

Celery at the center of your pipelines

Saying Goodbye to the Broker? WPipe vs. Celery

The context

Tired of configuring Redis or RabbitMQ just to run async tasks? 🐰 Celery is powerful, but its architecture requires an external "Broker" that adds latency and operational complexity. WPipe revolutionizes task execution with a "Broker-less" approach powered by its SQLite WAL persistence engine.

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="job", retry_count=3, retry_delay=1)
def job(data):
    return {"result": worker(data["task"])}

pipe = Pipeline(pipeline_name="brokerless")
pipe.set_steps([job])

# Async-safe, but no Redis/RabbitMQ required.
```

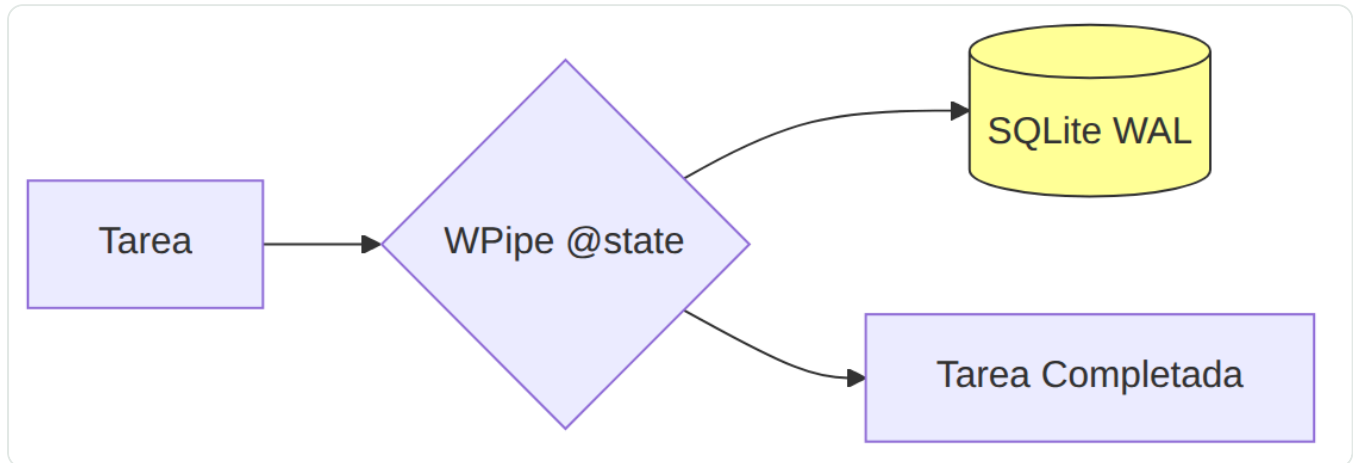
At a glance

Feature	WPipe	Celery
Infrastructure	Zero-Infra (Pure Python)	Broker (Redis/RabbitMQ)
RAM Usage	< 50MB	> 200MB
Resilience	Native (SQLite WAL)	Broker-dependent

Feature	WPipe	Celery
Setup Time	< 1 minute	Hours

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore Celery workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.